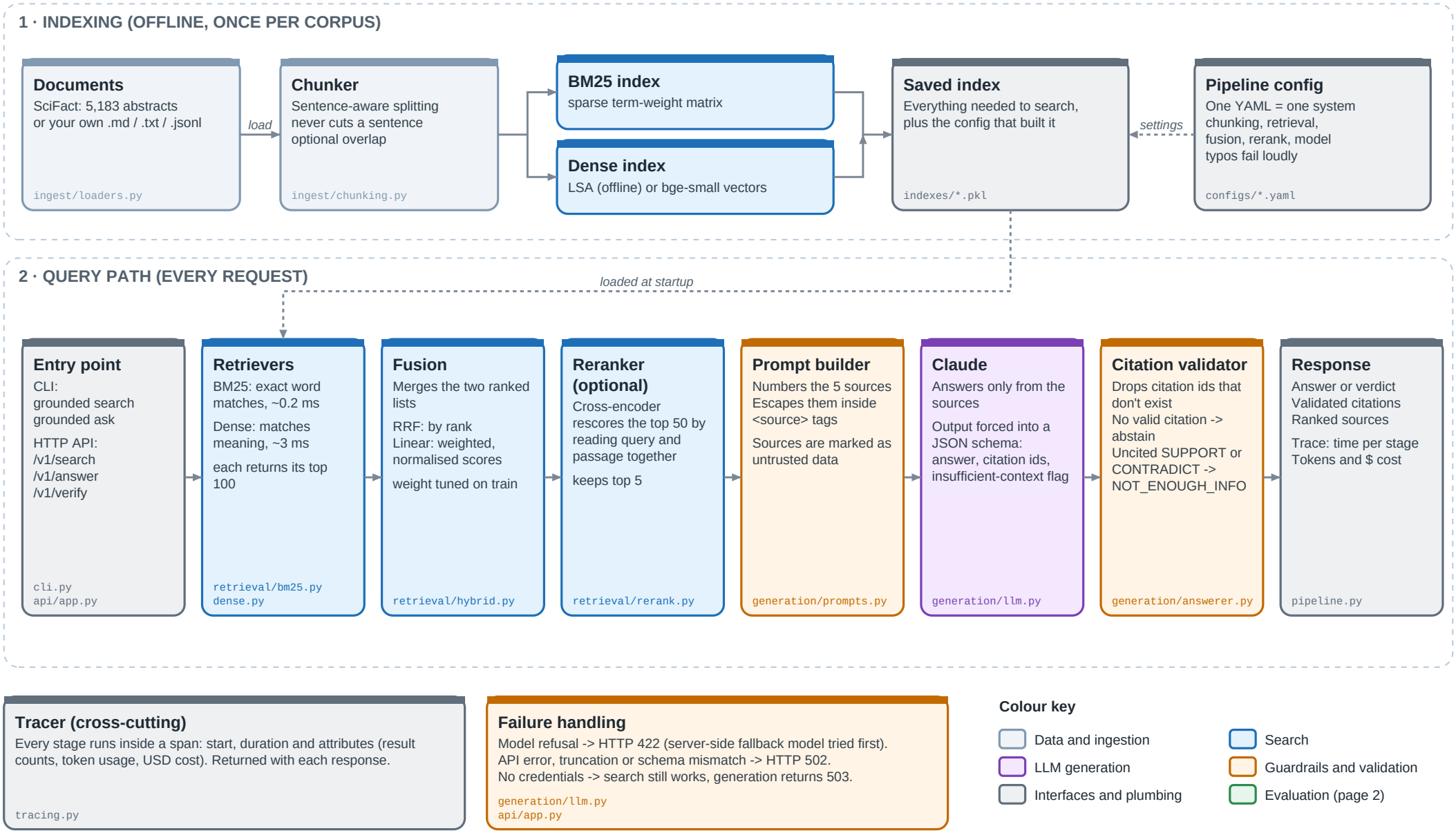
grounded-rag: how the components fit together

Indexing runs once per corpus. The query path runs on every request. One YAML config controls every stage; a tracer records each one.



grounded-rag: how quality is measured and protected

Settings are tuned on the training claims and reported on separate dev claims. CI re-runs the benchmark on every push and blocks regressions.

Measured on dev (nDCG@10): BM25 0.662 vs published 0.665 (reproduced) · naive blend -0.084 (p<0.001) · tuned system +0.012 (p=0.023)
...of which the blend alone, vs tuned BM25: +0.006 (p=0.105, not significant). Separate paired tests isolate each change.

SciFact benchmark

Allen Institute for AI, 2020

5,183 paper abstracts

Train: 809 claims
used only for tuning

Dev: 300 claims
used only for reporting

Expert labels per claim:
relevant abstracts (qrels)
SUPPORT / CONTRADICT / NEI
evidence sentences

`eval/datasets.py`

A · RETRIEVAL QUALITY (FREE, NO API KEY)

train →

Tune on train

Grid search:
BM25 k1 and b
fusion method and weight

best values -> configs

`eval/tune.py`

→

Systems under test

One YAML per variant:
bm25, bm25_tuned,
rm3, chunk100,
dense_lsa,
hybrid rrf / tuned

`configs/*.yaml`

→

Evaluate on dev

Each of 300 claims:
search, then compare the
ranking with the expert labels

nDCG@10, MRR@10,
recall@10 / @100, latency

`eval/runner.py`
`metrics.py`

dev claims ↑

→

Statistics

95% bootstrap CI per metric

Paired bootstrap test: same
claims, two systems -> is the
gap real?

`eval/metrics.py`

→

Reports

Markdown table
+ per-query JSON

--baseline picks
the comparison

`reports/`

B · END-TO-END ANSWER QUALITY (NEEDS ANTHROPIC_API_KEY)

labels →

Claim verification on dev

Retrieve -> Claude -> verdict, scored against the expert labels. No AI judge is needed.

`grounded eval verify`
`eval/runner.py`

→

What it reports

Verdict accuracy and macro-F1 vs the majority-class baseline (0.413)
Citation precision and recall vs gold evidence
Evidence-in-context rate, $/claim, latency

`eval/runner.py`

Your own documents

No expert labels? Claude writes test questions (synth), then a judge checks each answer claim by claim (faithfulness).

`eval/synth.py`
`judge.py`

C · CONTINUOUS INTEGRATION (GITHUB ACTIONS, EVERY PUSH AND PR)

1. Unit tests

57 pytest tests on Python 3.11 and 3.12, plus lint
8-document fixture corpus
Fake LLM and mock HTTP server: free, repeatable, no key

`tests/`

pass →

2. Re-run benchmark

Download SciFact
Evaluate bm25, bm25_tuned and hybrid_lsa_tuned on dev
Table added to the job summary

`.github/workflows/ci.yml`

scores →

3. Quality gate

Compares each metric with its floor:
bm25_tuned nDCG@10 >= 0.655, recall@100 >= 0.89, ...
Raise floors when quality improves

`eval/thresholds.yaml`

→ **PASS: change can merge**

→ **FAIL: metric below floor, merge blocked**

grounded-rag: component reference

Where each part lives and why it is built that way. Paths are relative to src/grounded/ unless shown otherwise.

Component	File	What it does	Key design decision
Loaders	`ingest/loaders.py`	Reads JSONL or folders of .md/.txt into documents	Same Document type for SciFact and your own files
Chunker	`ingest/chunking.py`	Splits documents into passages of up to N words	Never splits a sentence, so citations map back to whole sentences
BM25	`retrieval/bm25.py`	Keyword search over a precomputed sparse matrix	Written from scratch; reproduces the BEIR baseline (0.662 vs 0.665)
Dense retriever	`retrieval/dense.py`	Finds passages with similar meaning	LSA works offline for CI; bge-small is one install away
Fusion	`retrieval/hybrid.py`	Merges the BM25 and dense result lists	RRF needs no score calibration; weights are tuned on train only
Reranker	`retrieval/rerank.py`	Re-scores the top 50 with a cross-encoder	Optional: it costs latency, and the benchmark shows when it pays off
Prompt builder	`generation/prompts.py`	Formats numbered, escaped sources	Retrieved text is untrusted: fenced, escaped, declared as data
Claude client	`generation/llm.py`	Calls Claude with a JSON-schema output	Structured output; refusal fallback; cost from real token usage
Citation validator	`generation/answerer.py`	Checks every cited source id	Never trusts the model: invalid ids dropped, uncited verdicts downgraded
Pipeline + tracer	`pipeline.py, tracing.py`	Runs the stages and times each one	Every response carries latency, tokens and $
API and CLI	`api/app.py, cli.py`	HTTP endpoints and terminal commands	Search works without an API key; generation degrades to 503
Metrics and stats	`eval/metrics.py`	nDCG, MRR, recall, CIs, paired tests	Paired tests over the same queries detect small real differences
Runners and tuning	`eval/runner.py, tune.py`	Benchmark runs and grid search	Tune on train, report on dev: no peeking at the test answers
Judge and synth	`eval/judge.py, synth.py`	Eval for corpora without labels	AI grading AI: hand-check ~50 judgements before trusting it
Quality gate	`eval/gate.py, thresholds.yaml`	Fails CI when a metric drops	Floors only go up, unless there is a written reason

Life of one request

1. **Entry.** `grounded ask "Does vitamin D reduce fracture risk?"` loads the saved index and the config that built it.
2. **Retrieve.** BM25 and the dense retriever each return their top 100 passages. Fusion merges them into one ranked list, and the top 5 are kept (after reranking, if enabled).
3. **Prompt.** The 5 passages are numbered [1]-[5], escaped and wrapped in <source> tags, with an instruction to treat them as data and answer only from them.
4. **Generate.** Claude returns JSON that must match the schema: the answer text with [n] markers, the list of cited numbers, and an insufficient-context flag.
5. **Validate.** Cited numbers are checked against the 5 sources. Invalid ones are dropped, and an answer left with no valid citation is marked as an abstention.
6. **Respond.** The answer, resolved citations (document id, title, snippet), the ranked sources and a trace with time, tokens and $ per stage come back as one JSON object.